C语言内存函数详解

深入理解内存操作，提升编程内功

1. memcpy

```
void * memcpy ( void * destination, const void * source, size_t num );
```

功能：

- memcpy是完成内存块拷贝的，不关注内存中存储的数据是啥。
- 函数memcpy从source的位置开始向后复制num个字节的数据到destination指向的内存位置。
- 如果source和destination有任何的重叠，复制的结果都是未定义的。（内存重叠的情况使用memmove就行）
- memcpy的使用需要包含头文件`<string.h>`。

参数：

- **destination**：指针，指向目标空间，拷贝的数据存放在这里。
- **source**：指针，指向源空间，要拷贝的数据从这里来。
- **num**：要拷贝的数据占据的字节数。

返回值：

拷贝完成后，返回目标空间的起始地址。

1.1 代码演示

```
#include <stdio.h>
#include <string.h

int main()
{
    int arr1[] = { 1,2,3,4,5,6,7,8,9,10 };
    int arr2[10] = { 0 };
    memcpy(arr2, arr1, 20);
    int i = 0;
    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr2[i]);
    }
    return 0;
}
```

```
1 2 3 4 5 0 0 0 0 0
```

注意：使用 memcpy拷贝内存重叠的行为是未定义的，也就是说可能拷贝结果没问题，也可能在某种情况下会有问题。

memcpy

```
void * memcpy ( void * destination, const void * source, size_t num );
```

Copy block of memory

Copies the values of **num** bytes from the location pointed to by **source** directly to the memory block pointed to by **destination**.

The underlying type of the objects pointed to by both the **source** and **destination** pointers are irrelevant for this function; The result is a binary copy of the data.

The function does not check for any terminating null character in **source** - it always copies exactly **num** bytes.

To avoid overflows, the size of the arrays pointed to by both the **destination** and **source** parameters, shall be at least **num** bytes, and should not overlap (for overlapping memory blocks, memmove is a safer approach).

示例：

```
#include <stdio.h>
#include <string.h>

int main()
{
    int arr[] = { 1,2,3,4,5,6,7,8,9,10 };

    memcpy(&arr[2], &arr[0], 5 * sizeof(int));

    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

在此时重叠的情况下，预期结果可能是：(1,2,1,2,1,2,3,9,10)。

虽然运行结果是正确的，但是官方还是以为这种属于未定义的行为。这种情况下建议使用memmove

2. memmove

```
void * memmove ( void * destination, const void * source, size_t num );
```

功能：

- memmove函数也是完成内存块拷贝的。
- 和memcpy的差别就是memmove函数处理的源内存块和目标内存块是可以重叠的。
- memmove的使用需要包含头文件`<string.h>`。

参数：

- **destination**：指针，指向目标空间，拷贝的数据存放在这里。
- **source**：指针，指向源空间，要拷贝的数据从这里来。
- **num**：要拷贝的数据占据的字节数。

返回值：

拷贝完成后，返回目标空间的起始地址。

memmove `<string>`

```
void * memmove ( void * destination, const void * source, size_t num );
```

Move block of memory

Copies the values of **num** bytes from the location pointed to by source to the memory block pointed by destination.
Copying takes place as if an intermediate buffer were used, allowing the destination and source to overlap.

The underlying type of the objects pointed by both the source and destination pointers are irrelevant for this function;
The result is a binary copy of the data.

The function does not check for any terminating null character in source - it always copies exactly num bytes.

To avoid overflows, the size of the arrays pointed by both the destination and source parameters, shall be at least num bytes.

2.1 代码演示

```
#include <stdio.h>
#include <string.h>

int main()
{
    int arr[] = { 1,2,3,4,5,6,7,8,9,10 };

    memmove(&arr[2], &arr[0], 5 * sizeof(int));

    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

输出的结果：

```
1 2 1 2 3 4 5 9 10
```

3. memset

```
void * memset ( void * ptr, int value, size_t num );
```

功能：

- memset函数是用来设置内存块的内容的，将内存中指定长度的空间设置为特定的内容。
- memset的使用需要包含头文件`<string.h>`。

参数：

- **ptr**：指针，指向要设置的内存空间，也就是存放了要设置的内存空间的起始地址。
- **value**：要设置的值，函数将会把value值转换成unsigned char的数据进行设置的。也就是以字节为单位来设置内存空间。
- **num**：要设置的内存长度，单位是字节。

返回值：

返回的是要设置的内存空间的起始地址。

3.1 代码演示

```
#include <stdio.h>
#include <string.h>
int main ()
{
    char str[] = "hello world";
    memset (str,'x',6);
    printf(str);
    return 0;
}
```

输出的结果：

```
xxxxxxworld
```

3.2 总结：

当有一块内存空间需要设置内容的时候，就可以使用memset函数，值得注意的是memset函数对内存单元的设置是以字节为单位的。

4. memcmp

```
int memcmp ( const void * ptr1, const void * ptr2, size_t num );
```

功能：

- 比较指定的两块内存块的内容，比较从ptr1和ptr2指针指向的位置开始，向后的num个字节。
- memcmp的使用需要包含头文件`<string.h>`。

参数：

- **ptr1**：指针，指向一块待比较的内存块。
- **ptr2**：指针，指向另外一块待比较的内存块。
- **num**：指定的比较长度，单位是字节。

返回值：

Return Value

Returns an integral value indicating the relationship between the content of the memory blocks:

return value	indicates
`<0`	the first byte that does not match in both memory blocks has a lower value in *ptr1* than in *ptr2* (if evaluated as unsigned char values)
`0`	the contents of both memory blocks are equal
`>0`	the first byte that does not match in both memory blocks has a greater value in *ptr1* than in *ptr2* (if evaluated as unsigned char values)

4.1 代码演示

```
#include <stdio.h>
#include <string.h>

int main()
{
    char buffer[] = "DWgaOtP12df0";
    char buffer2[] = "DWGAOtP12df0";
    int n;
    n = memcmp(buffer1, buffer2, sizeof(buffer1));

    if (n > 0)
    {
        printf("%s' 大于 '%s'.\n", buffer1, buffer2);
    }
    else if (n < 0)
    {
        printf("%s' 小于 '%s'.\n", buffer1, buffer2);
    }
    else
    {
        printf("%s' 和 '%s' 一样.\n", buffer1, buffer2);
    }
    return 0;
}
```

```
DWgaOtP12df0' 大于 'DWGAOtP12df0'.
```

4.2 总结

如果要比较以块内存空间的数据的大小，可以使用memcmp函数，这个函数的特点就是可以指定比较长度。memcmp函数是通过逐回逐告知大小关系的。

📋 本节总结

- **memcpy**: 用于内存拷贝，速度快，但不能处理源和目标内存重叠的情况。
- **memmove**: 同样用于内存拷贝，但能安全处理内存重叠，比memcpy更健壮。
- **memset**: 用于将一段内存区域的内容设置为指定值，操作单位是字节。
- **memcmp**: 用于比较两块内存区域的内容，可指定比较的字节数。
- 所有这些函数都在 `string.h` 头文件中声明。